动态规划-子序列问题

码上总结

动态规划

动态规划概念

通过把原问题分解为简单的子问题来解决复杂问题，把每个子问题的最优解保存，用于求解最终最优解，已经算过的不会重复计算，从而提高解决问题的效率。

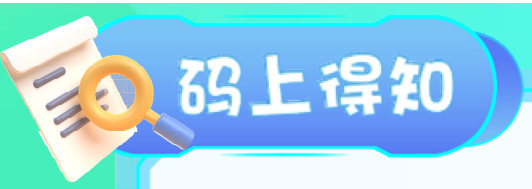
动态规划基本要素

1.最优子结构
2.无后效性
3.重叠子问题

动态规划求解步骤

1.确定状态转移方程
2.初始化状态
3.状态转移

最大子段和



最大子段和概念

如下序列：

1 -2 5 3 -4 5

子段是从序列中截取某段连续的序列 4 2 8

将子段中数字相加

得到子段和

1 -2 5 -2 5 3 -4 1 -2 5 3 -4 5

可以有多少个子段，就会有多少个子段和，所有的子段和中，最大的，称为最大子段和

这些都属于原序列的子段

非连续序列，都不算子段

而这些：

1 -2 5 -4 5 3 5 -2 5 -4 5

解码时刻

温度统计

【问题描述】

某地区温度非常不稳定，每天的最高温都会有很大波动，现在需要求出一段日期里的温度总和，这段日期必须是连续的，长度可以是1~n，这个温度总和要尽量大。

【输入格式】

两行，第一行一个整数n，表示记录的温度天数，第二行为每天的温度x，用空格隔开。（1≤n≤100，-10≤x≤10）

【输出格式】

一个整数，表示最大的某几天连续温度的总和。

【输入样例】

6

1 -2 5 3 -4 5

【输出样例】

9

样例分析

【输入样例】
6
1 -2 5 3 -4 5
【输出样例】
9

第一行数字6，表示序列中有6个数字

1 -2 5 3 -4 5

子段必须连续

5 + 3 + (-4) + 5 = 9

数字9为最大子段和，最终输出9



问题分析

最大子段和问题依然可以用动态规划处理。

数字序列：	1	-2	5	3	-4	5
最大子段和：	1	-1	5	8	4	9

某个数字为结尾时形成的最大子段和

从最小规模问题出发，一步步求解最大子段和。

从第2个位置开始每个位置的最优解面临两种选择

和左侧相邻子段结合　　加左侧值

自己单独形成一个子段　　不加左侧值

子段必须连续，所以只需考虑是否加左侧挨着的值。

问题分析

最大子段和问题依然可以用动态规划处理。

数字序列：	1	-2	5	3	-4	5
最大子段和：	1	-1	5	8	4	9

某个数字为结尾时形成的最大子段和

状态转移方程

当前子段和 = max(当前数字，左侧子段和 + 当前数字)

数组a存储数字序列，dp存储以不同数字为结尾的最大子段和。i位置数字结尾的最大子段和为:

dp[i] = max(a[i] , dp[i-1] + a[i])

解码时刻

解题步骤

1.数据输入

```
//n表示数字序列长度,a存储数字序列
int n,a[101];
cin>>n;
for(int i=1;i<=n;i++){
    cin>>a[i];
    ... ...
}
```

解码时刻

解题步骤

1.数据输入

```
//n表示数字序列长度,a存储数字序列,dp存子段和
int n,a[101],dp[101];
cin>>n;
for(int i=1;i<=n;i++){
    cin>>a[i];
```

2.初始化dp数组

```
    if(i==1) dp[i]=a[i];
```

3.状态转移

```
    else dp[i]=max(a[i],dp[i-1]+a[i]);
}
```

数字序列：	1	-2	5	3	-4	5
最大子段和：	1	-1	5	8	4	9

用序列中第一个数进行初始化

max(max(max(max(max(1,-2)+5),3)+-4)+5)

解码时刻

解题步骤

1.数据输入

```
//n表示数字序列长度,a存储数字序列,dp存子段和,ans初始化成极小值
int n,a[101],dp[101],ans=-1000;
cin>>n;
for(int i=1;i<=n;i++){
    cin>>a[i];
```

2.初始化dp数组

```
    if(i==1) dp[i]=a[i];
```

3.状态转移

```
    else dp[i]=max(a[i],dp[i-1]+a[i]);
    //保存最大子段和
    ans=max(ans,dp[i]);
}
```

4.输出结果

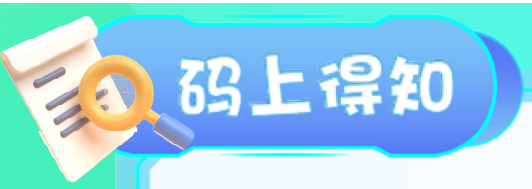
```
cout<<ans;
```

数字序列：	1	-2	5	3	-4	5
最大子段和：	1	-1	5	8	4	9

完整代码

```
#include<bits/stdc++.h>
using namespace std;
int n,a[101],dp[101],ans=-1000;
int main(){
    //输入数据
    cin>>n;
    for(int i=1;i<=n;i++){
        cin>>a[i];
        //初始化dp数组
        if(i==1) dp[i]=a[i];
        //状态转移
        else dp[i]=max(a[i],dp[i-1]+a[i]);
        //保存最大子段和
        ans=max(ans,dp[i]);
    }
    //输出结果
    cout<<ans;
    return 0;
}
```

最长上升子序列



最长上升子序列

有数字序列：

5 3 9 11 4 2 15

什么是子序列？

5 3 9 15

9 4

2

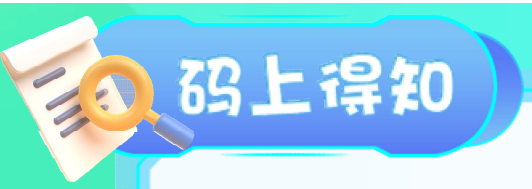
5 3 9 11 4 2 15

. . .

这些都属于原序列的子序列

原数字序列去掉一些数字形成的新序列

原数字序列本身也算子序列



最长上升子序列

有数字序列：

5 3 9 11 4 2 15

什么是上升子序列？

5 11 15

3 9 11 15

4 15

5 9 11 15

. . .

子序列中

i位置值 < i+1位置值

码上得知

最长上升子序列

有数字序列：

5 3 9 11 4 2 15

什么是上升子序列？

5 11 15

3 9 11 15

4 15

5 9 11 15

. . .

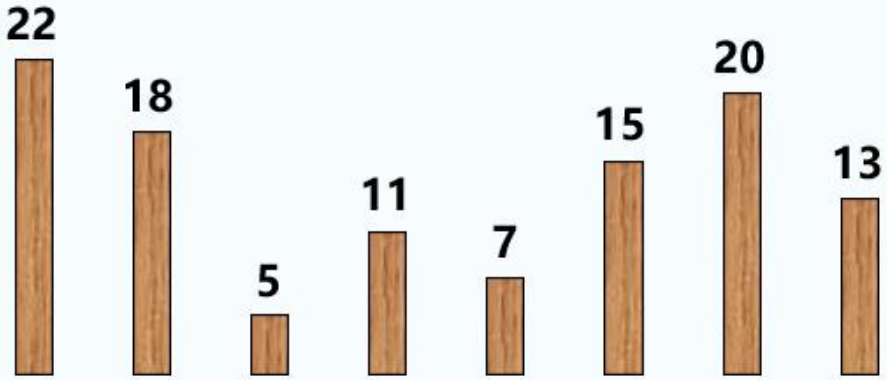
在所有上升子序列中
最长的为最长上升子序列

解码时刻

制作信箱

【问题描述】

小童想亲手制作一个木质的信箱，于是他到林中收集木棍，在收集木棍的过程中有一个原则，新收集的木棍要比上一个收集的木棍要长，现有如下木棍，从左到右顺序收集的话，小童最多可以收集多少根木棍。





解码时刻

制作信箱

【输入格式】

第一行一个整数n，表示木棍的数量。(1≤n≤1000)

第二行有n个正整数，用空格隔开，表示木棍的长度。(单根木棍长≤1e9)

【输出格式】

一个整数，表示可收集的最大木棍数量。

【输入样例】

8

22 18 5 11 7 15 20 13

【输出样例】

4

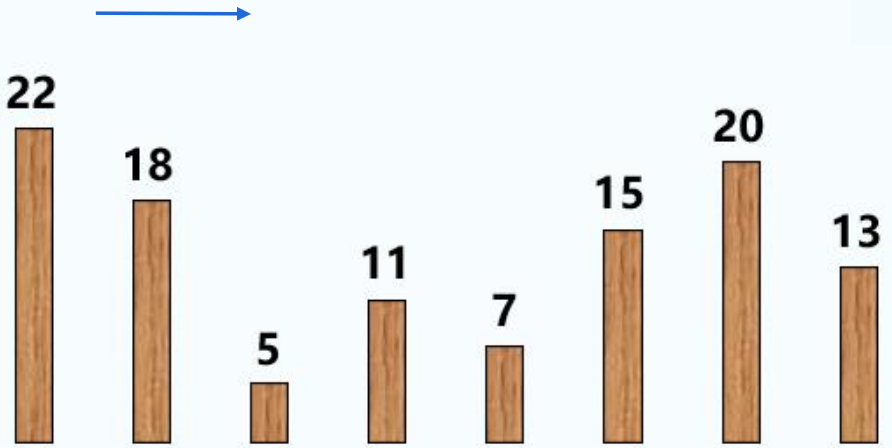
解码时刻

样例分析

【输入样例】
8
22 18 5 11 7 15 20 13
【输出样例】
4

第1次拿22，最多拿1根。
第1次拿18，最多拿2根。
第1次拿5，最多拿4根。
5、11、15、20
或
5、7、15、20
这是最多的拿法，结果为4。

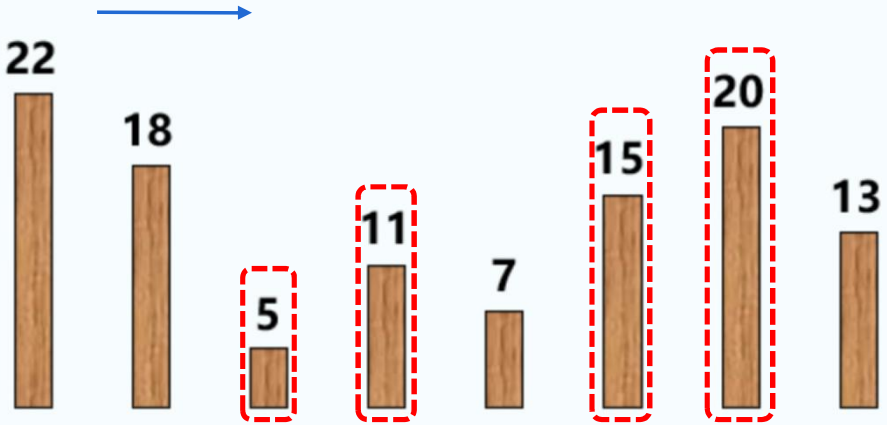
从左到右顺序选取，每次选取的木棍要比前一次选取的木棍要长。



解码时刻

问题分析

此题的本质是在序列中找到最长的上升序列。



最长上升序列长度为4

最长上升序列问题属于经典的动态规划问题。

解码时刻

问题分析

先把问题简化，从1根木头开始，把不同木头数量的最大上升序列长度都求出来:

一根木头

22



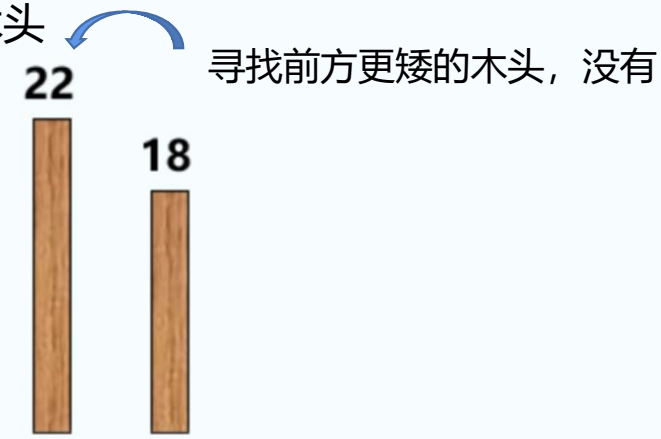
最大上升序列长度：**1**

解码时刻

问题分析

先把问题简化，从1根木头开始，把不同木头数量的最大上升序列长度都求出来:

2根木头



寻找前方更矮的木头，没有

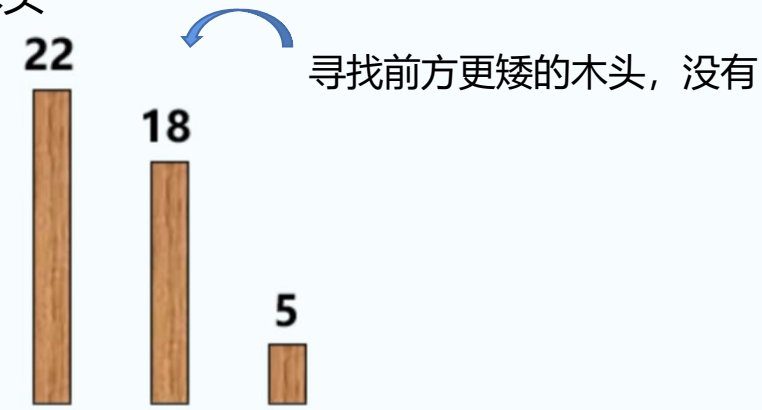
最大上升序列长度：1　　1

解码时刻

问题分析

先把问题简化，从1根木头开始，把不同木头数量的最大上升序列长度都求出来:

3根木头



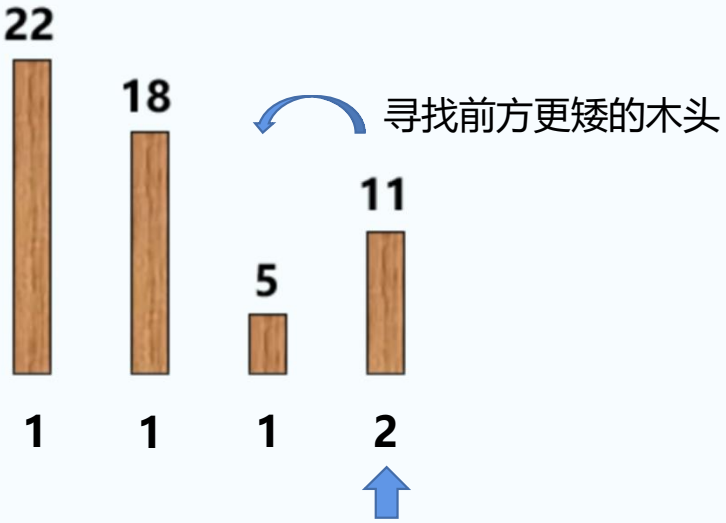
最大上升序列长度：1　1　1

解码时刻

问题分析

先把问题简化，从1根木头开始，把不同木头数量的最大上升序列长度都求出来:

4根木头

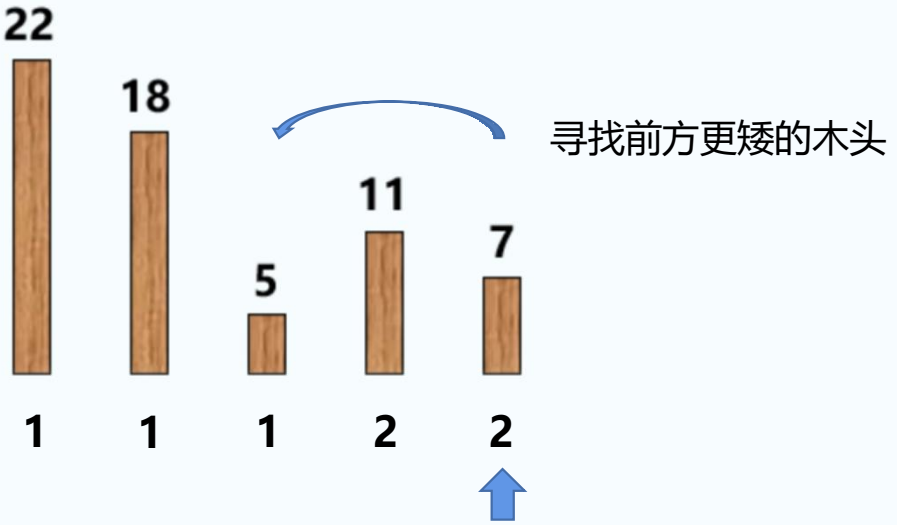


解码时刻

问题分析

先把问题简化，从1根木头开始，把不同木头数量的最大上升序列长度都求出来:

5根木头

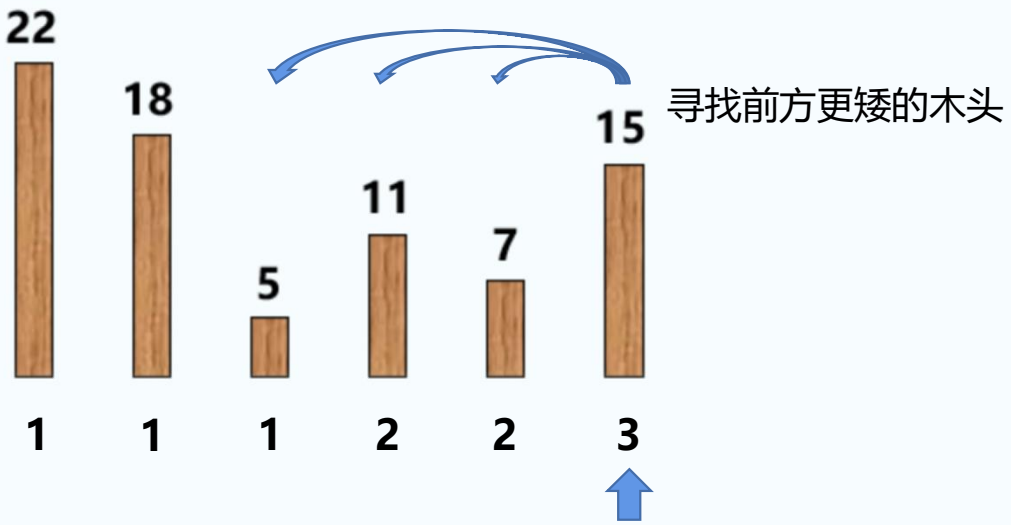


解码时刻

问题分析

先把问题简化，从1根木头开始，把不同木头数量的最大上升序列长度都求出来:

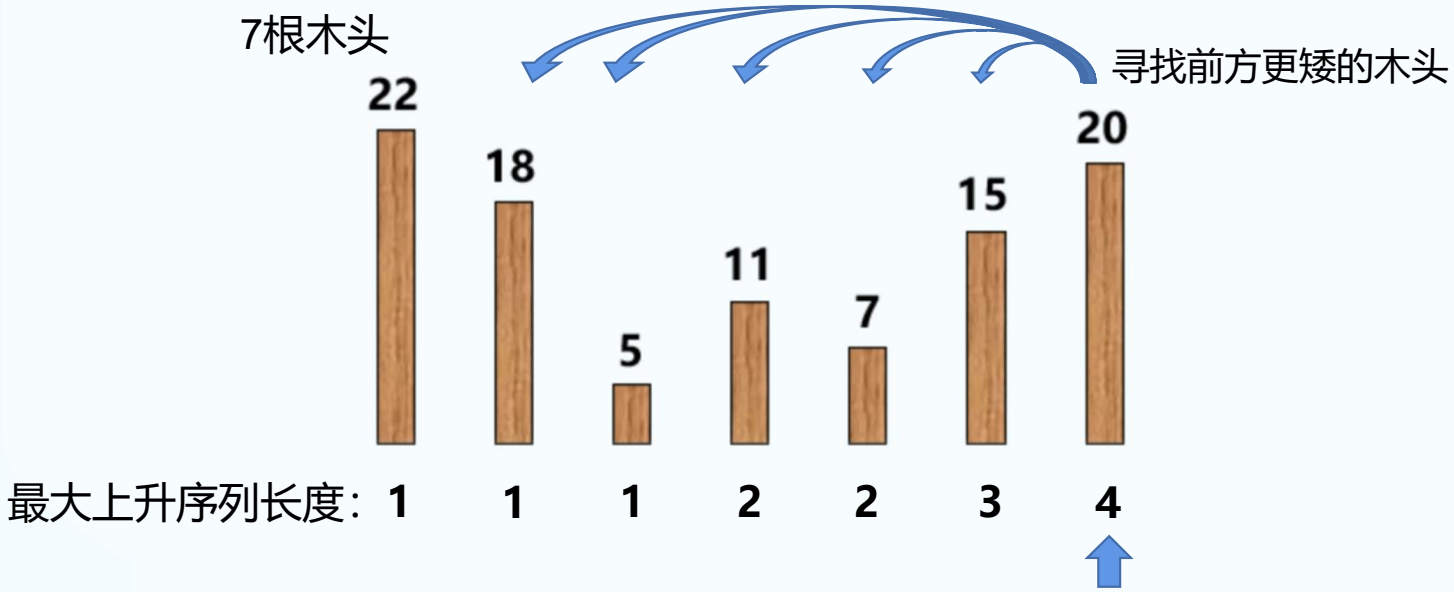
6根木头



解码时刻

问题分析

先把问题简化，从1根木头开始，把不同木头数量的最大上升序列长度都求出来:

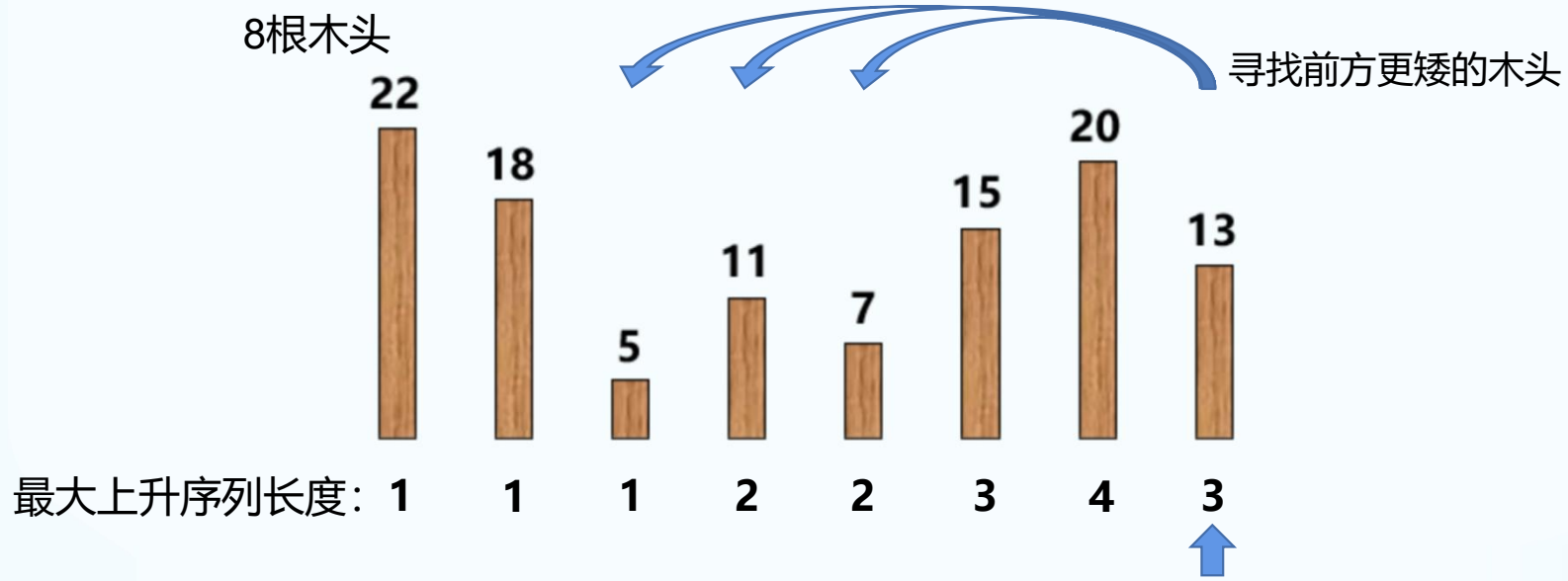


解码时刻

问题分析

先把问题简化，从1根木头开始，把不同木头数量的最大上升序列长度都求出来:

8根木头

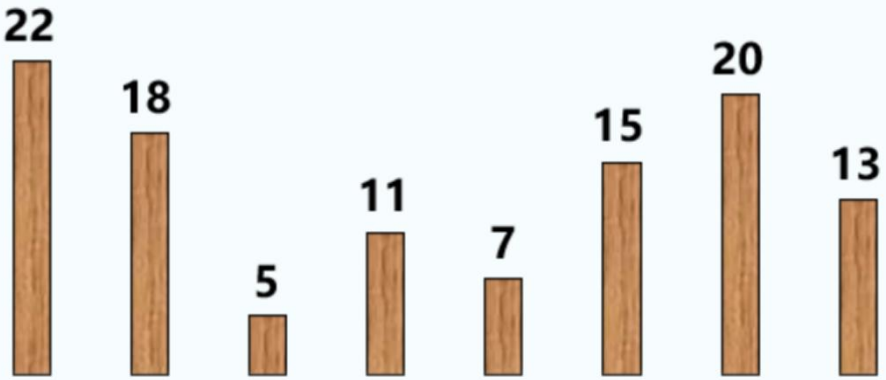


解码时刻

问题分析

先把问题简化，从1根木头开始，把不同木头数量的最大上升序列长度都求出来:

8根木头



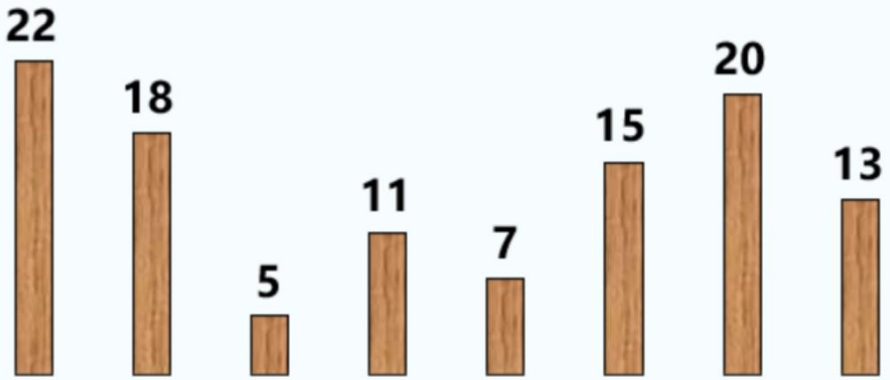
最大上升序列长度：1 1 1 2 2 3 4 3

从所有的上升长度中选择最大值作为最终解

解码时刻

问题分析

当前问题具备哪些动规要素？



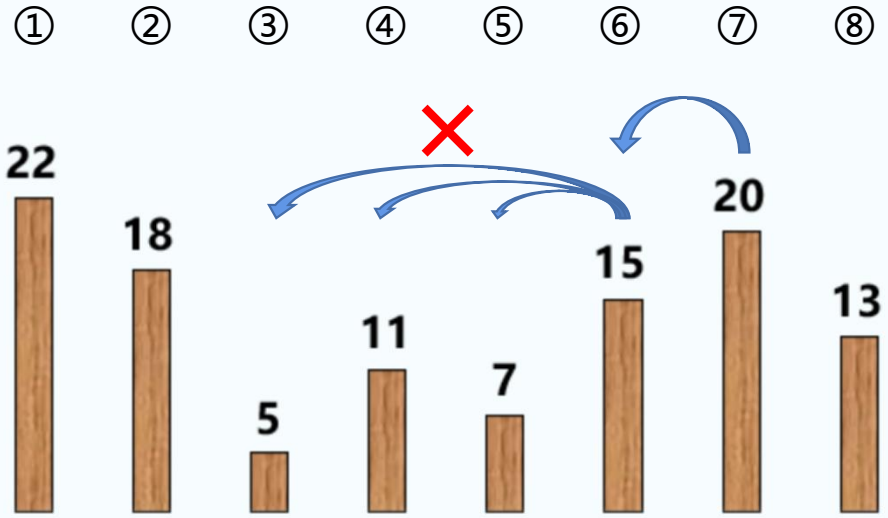
最大上升序列长度：1 1 1 2 2 3 4 3

1.每一步的解都是当前木头作为终点时的最优解，由小问题最优解向更大问题最优解转移，最优子结构。

解码时刻

问题分析

当前问题具备哪些动规要素？



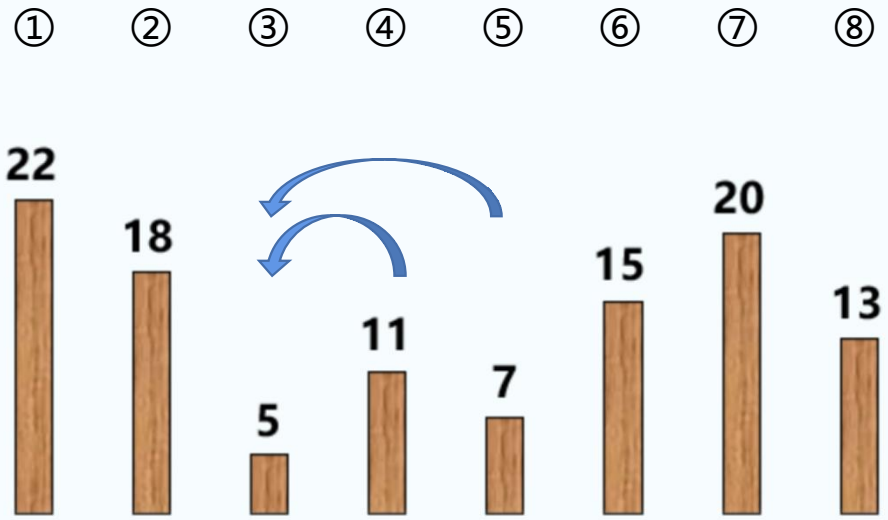
最大上升序列长度：1 1 1 2 2 3 4 3

2.例如求第7根木头的解，是用第6根木头的解计算而来，用不到第6根木头前一步的解，无后效性。

解码时刻

问题分析

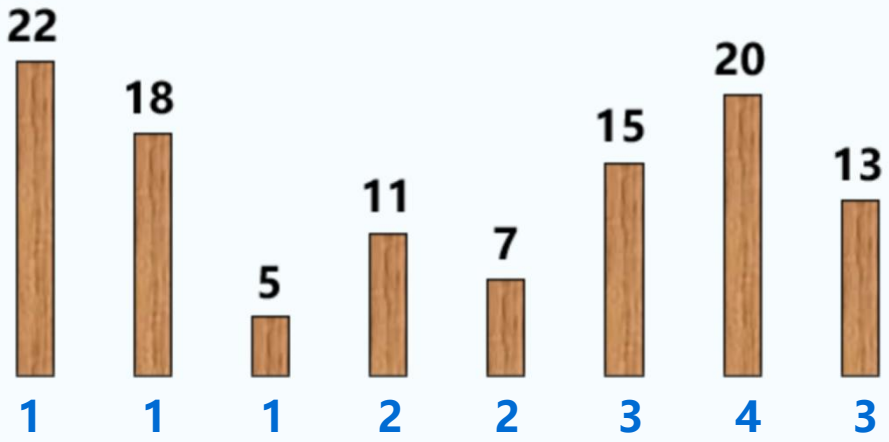
当前问题具备哪些动规要素？



3.子问题与子问题是相同的问题，并且子问题会被重复使用。
例如求第4根木头用到了第3根木头的解，求第5根木头也用到了第3根木头的解，重叠子问题。

问题分析

状态转移方程



状态转移方程：左侧可用的最大值 + 1 = 当前位置可拿数量

解码时刻

解题步骤

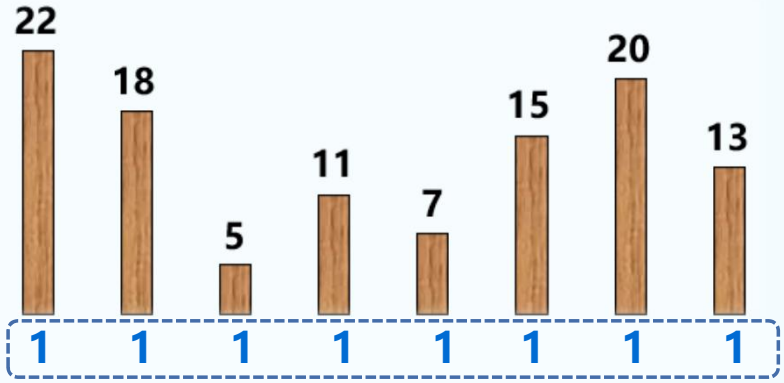
1.输入数据

```
int a[1001],n;  //定义在全局

cin>>n; //木棍数量
for (int i=1; i<=n; i++) {
    cin>>a[i]; //木棍长度
}
```

2.初始化状态

```
int dp[1001]; //定义在全局
for (int i=1; i<=n; i++) {
    dp[i]=1;
}
```



dp 数组存储不同位置最大上升子序列长度。
最小长度为1，所以全部初始化为1。

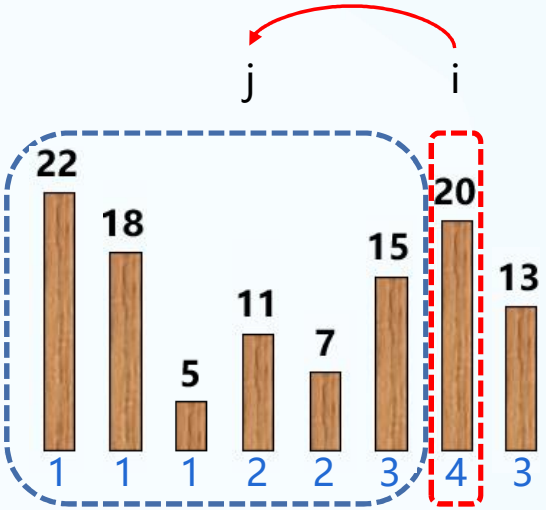
解题步骤

3.状态转移

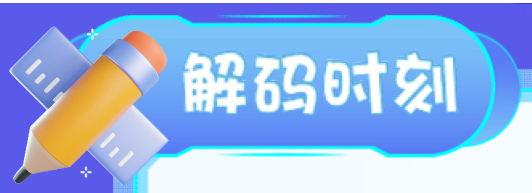
```
//遍历所有木棍
for(int i=1; i<=n; i++) {
    //遍历第i根木棍左侧的所有木棍
    for(int j=1; j<i; j++) {
        //判断木棍是否可用        状态转移方程
        if(a[j]<a[i]) dp[i]=max(dp[i],dp[j]+1);
    }
}
```

4.输出结果

```
int ans=0;  //存储结果
for(int i=1;i<=n;i++) {
    ans=max(ans,dp[i]);  //保存最大值
}
cout<<ans;
```



每次遍历1根木棍(i)，检查左侧所有木棍(j)是否有可用的，进行状态转移，留存最大值。



完整代码

```cpp
#include<bits/stdc++.h>
using namespace std;
int a[1001], dp[1001], n;
int main() {
    //输入
    cin>>n;
    for (int i=1; i<=n; i++) {
        cin>>a[i];
    }
    //初始化
    for (int i=1; i<=n; i++) {
        dp[i]=1;
    }
    for(int i=1; i<=n; i++) {
        for(int j=1; j<i; j++) {
            //状态转移
            if(a[j]<a[i]) {
                dp[i]=max(dp[i],dp[j]+1);
            }
        }
    }
    int ans=0;
    for(int i=1;i<=n;i++) {
        ans=max(ans,dp[i]);
    }
    //输出结果
    cout<<ans;
    return 0;
}
```

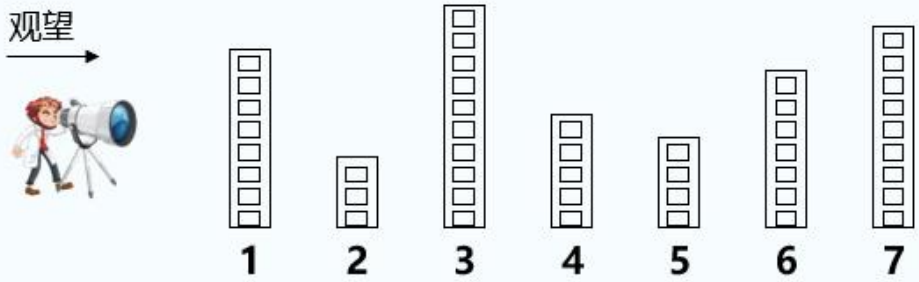
解码时刻

高低错落的大楼

【问题描述】

有n座大楼排在一条直线上，编号分别为1~n，每栋大楼的高度不同，你刚好站在这条直线上，从小编号的楼往大编号楼方向观望，如下图，如果某栋楼在高度上没有被挡住，那就可以被观望到，这里只考虑楼高不考虑其他因素。

可任意选择从某一栋楼前开始观望，观望方向只能从小编号往大编号方向观望，求最大可观望数量。



解码时刻

高低错落的大楼

【输入格式】

第一行一个整数n，表示楼的数量。(1≤n≤1000)

第二行n个正整数，用空格隔开，表示楼的高度。(独栋高度≤1e9)

【输出格式】

一个整数，表示最大的观望数量。

【样例输入】

7

65 28 87 45 30 63 70

【样例输出】

3

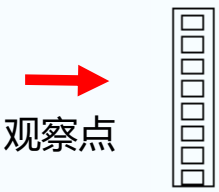
解码时刻

样例分析

【样例输入】
7
65 28 87 45 30 63 70
【样例输出】
3

当只有一栋楼时

1号楼



观望数量：　1

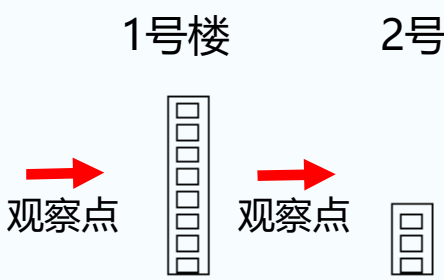
只可以看到1栋楼。

解码时刻

样例分析

【样例输入】
7
65 28 87 45 30 63 70
【样例输出】
3

当有两栋楼时，2号楼作为观看终点



可以看到1栋

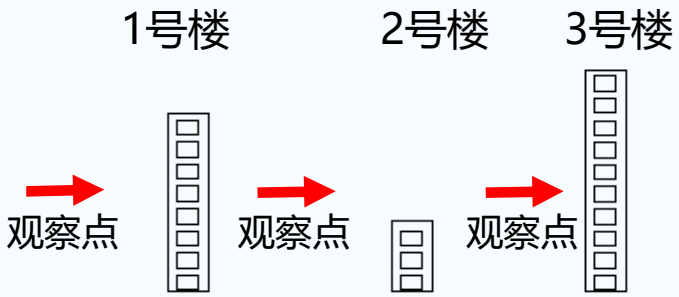
无法观望到2号楼

解码时刻

样例分析

【样例输入】
7
65 28 87 45 30 63 70
【样例输出】
3

当有三栋楼时，3号楼作为观看终点



可以看到1栋
可以看到2栋
可以看到2栋

解码时刻

样例分析

【样例输入】
7
65 28 87 45 30 63 70
【样例输出】
3

当有四栋楼时，4号楼作为观看终点



可以看到1栋

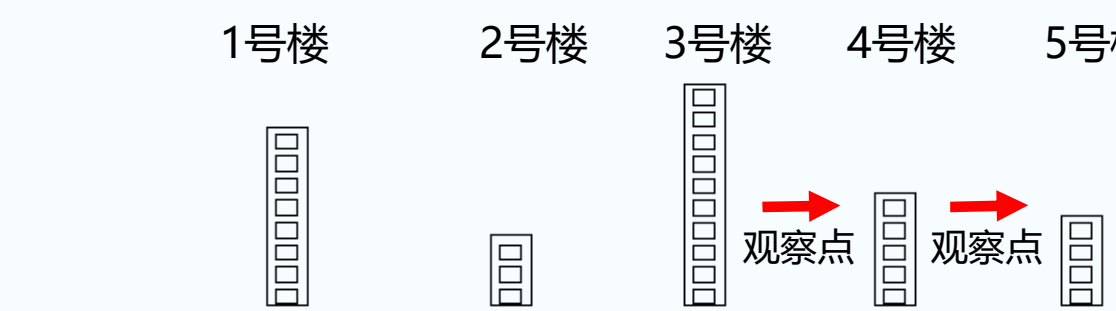
从3号楼开始看不到4号楼，不用再往左尝试

解码时刻

样例分析

【样例输入】
7
65 28 87 45 30 63 70
【样例输出】
3

当有五栋楼时，5号楼作为观看终点



可以看到1栋

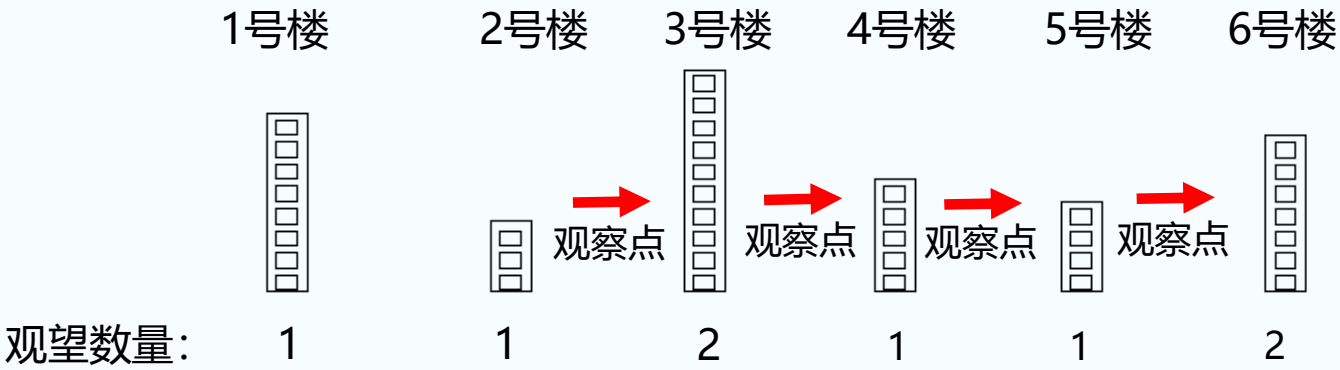
从4号楼开始看不到5号楼，不用再往左尝试

解码时刻

样例分析

【样例输入】
7
65 28 87 45 30 63 70
【样例输出】
3

当有六栋楼时，6号楼作为观看终点



可以看到1栋
可以看到2栋
可以看到2栋
从3号楼开始看不见6号楼，不用再往左尝试

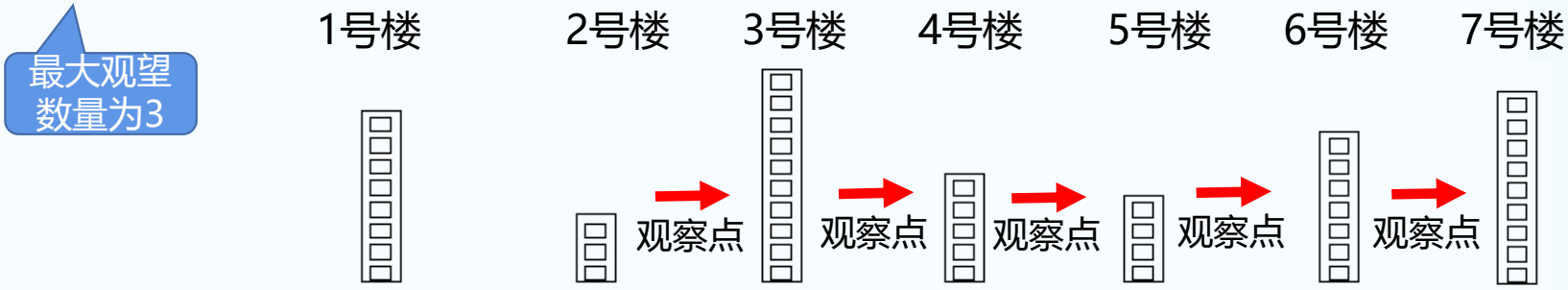
解码时刻

样例分析

【样例输入】
7
65 28 87 45 30 63 70
【样例输出】
3

最大观望数量为3

当有六栋楼时，6号楼作为观看终点



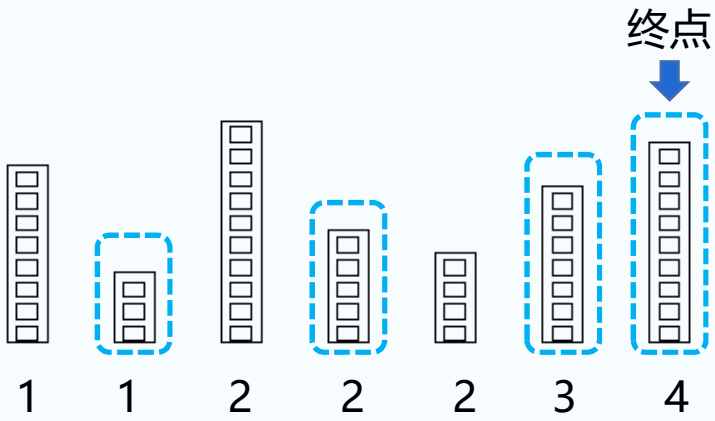
可以看到1栋
可以看到2栋
可以看到3栋
可以看到3栋

从3号楼开始看不到7号楼，
不用再往左尝试

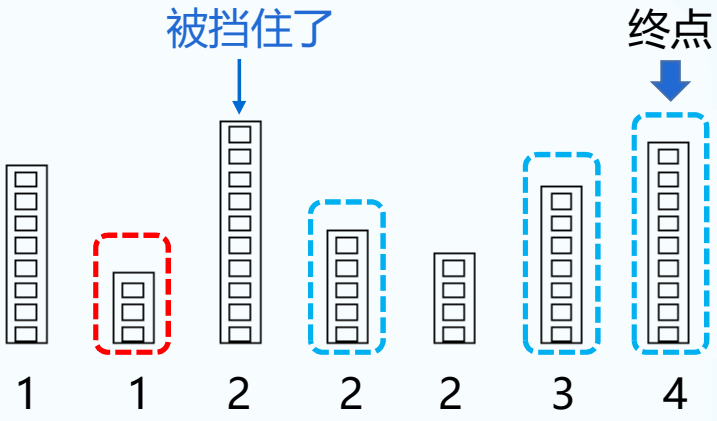
解码时刻

问题分析

与“制作信箱”问题的区别



制作信箱思路，这里最大上升子序列长度4。



此题情况，无法形成长度为4的上升序列。

```
if(a[j]<a[i]) dp[i]=max(dp[i],dp[j]+1);
```

终点左侧可用高度　　终点

两者状态转移方程类似，不同之处是**可用高度的范围**不一样

解码时刻

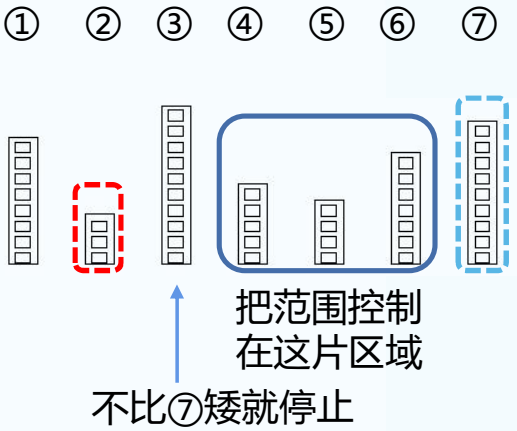
问题分析

调整“制作信箱”的状态转移代码

dp数组:存储以第i栋楼为终点的观望数量
a数组:存储楼高

```
//遍历楼高
for(int i=1; i<=n; i++){
    //遍历第i栋楼左侧所有楼高
    for(int j=1; j<i; j++){
        //状态转移
        if(a[j]<a[i]) dp[i]=max(dp[i],dp[j]+1);
    }
}
```





假设求⑦的观望数量，以上代码会把②
也算进来

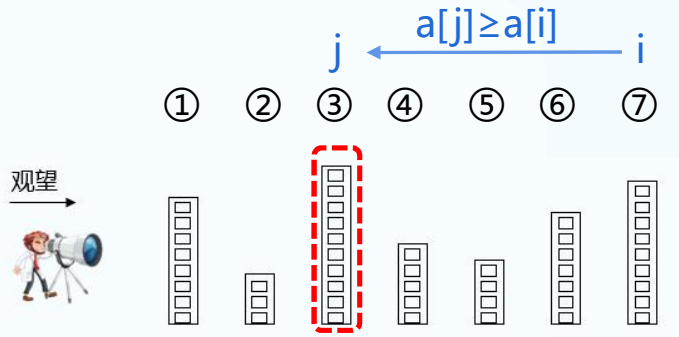
解码时刻

问题分析

调整“制作信箱”的状态转移代码

dp数组:存储以第i栋楼为终点的观望数量
a数组:存储楼高

```
//遍历楼高
for(int i=1; i<=n; i++){
    //遍历第i栋楼左侧所有楼高
    for(int j=i-1; j>=1; j--){
        //状态转移
        if(a[j]<a[i]) dp[i]=max(dp[i],dp[j]+1);
        else break;
    }
}
```



内层循环**从右至左遍历**，当大于等于第 i 栋高度，结束当前循环(内层循环)

解码时刻

问题分析

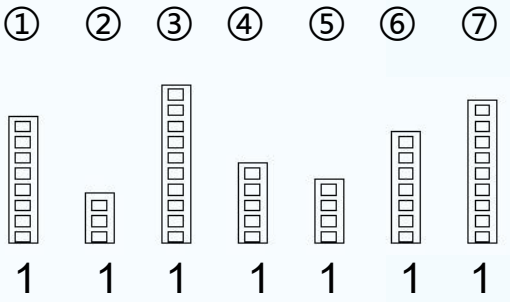
调整“制作信箱”的状态转移代码

dp数组:存储以第i栋楼为终点的观望数量
a数组:存储楼高

```
//遍历楼高
for(int i=1; i<=n; i++){
    //遍历第i栋楼左侧所有楼高
    for(int j=i-1; j>=1; j--){
        //状态转移
        if(a[j]<a[i]) dp[i]=max(dp[i],dp[j]+1);
        else break;
    }
}
```



执行流程演示



初始化为1

内层循环**从右至左遍历**，当大于等于第 i 栋高度，结束当前循环(内层循环)

解码时刻

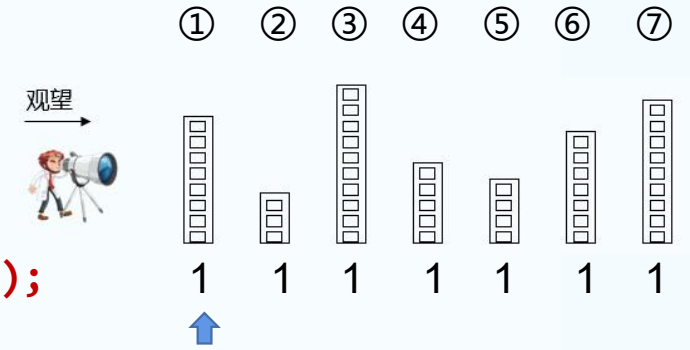
问题分析

调整“制作信箱”的状态转移代码

dp数组:存储以第i栋楼为终点的观望数量
a数组:存储楼高

```
//遍历楼高
for(int i=1; i<=n; i++){
    //遍历第i栋楼左侧所有楼高
    for(int j=i-1; j>=1; j--){
        //状态转移
        if(a[j]<a[i]) dp[i]=max(dp[i],dp[j]+1);
        else break;
    }
}
```

执行流程演示



内层循环**从右至左遍历**，当大于等于第 i 栋高度，结束当前循环(内层循环)

解码时刻

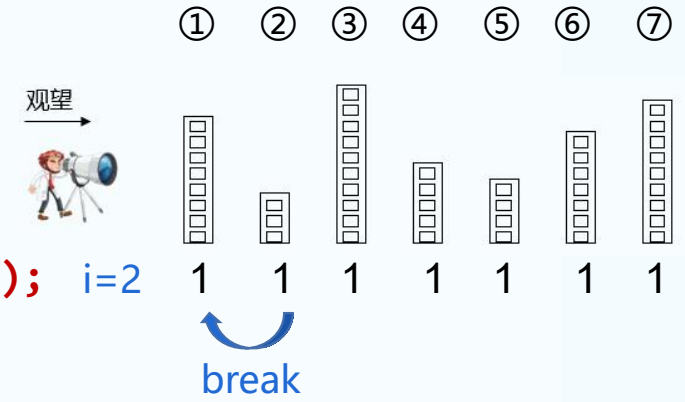
问题分析

调整“制作信箱”的状态转移代码

dp数组:存储以第i栋楼为终点的观望数量
a数组:存储楼高

```
//遍历楼高
for(int i=1; i<=n; i++){
    //遍历第i栋楼左侧所有楼高
    for(int j=i-1; j>=1; j--){
        //状态转移
        if(a[j]<a[i]) dp[i]=max(dp[i],dp[j]+1);
        else break;
    }
}
```

执行流程演示



内层循环**从右至左遍历**，当大于等于第 **i** 栋高度，结束当前循环(内层循环)

解码时刻

问题分析

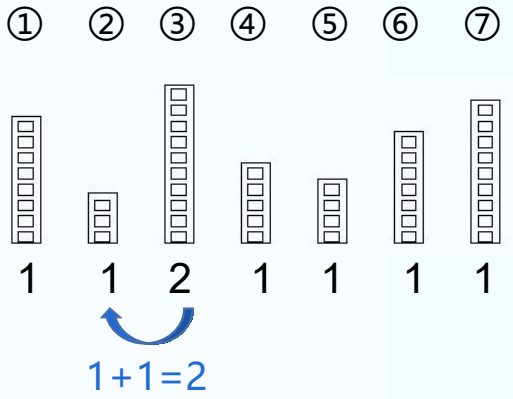
调整“制作信箱”的状态转移代码

dp数组:存储以第i栋楼为终点的观望数量
a数组:存储楼高

```
//遍历楼高
for(int i=1; i<=n; i++){
    //遍历第i栋楼左侧所有楼高
    for(int j=i-1; j>=1; j--){
        //状态转移
        if(a[j]<a[i]) dp[i]=max(dp[i],dp[j]+1);
        else break;
    }
}
```



执行流程演示



内层循环**从右至左遍历**，当大于等于第 i 栋高度，结束当前循环(内层循环)

解码时刻

问题分析

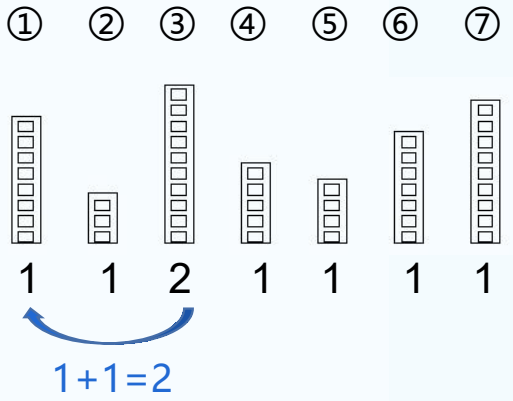
调整“制作信箱”的状态转移代码

dp数组:存储以第i栋楼为终点的观望数量
a数组:存储楼高

```
//遍历楼高
for(int i=1; i<=n; i++){
    //遍历第i栋楼左侧所有楼高
    for(int j=i-1; j>=1; j--){
        //状态转移
        if(a[j]<a[i]) dp[i]=max(dp[i],dp[j]+1);
        else break;
    }
}
```



执行流程演示



内层循环**从右至左遍历**，当大于等于第 **i** 栋高度，结束当前循环(内层循环)

解码时刻

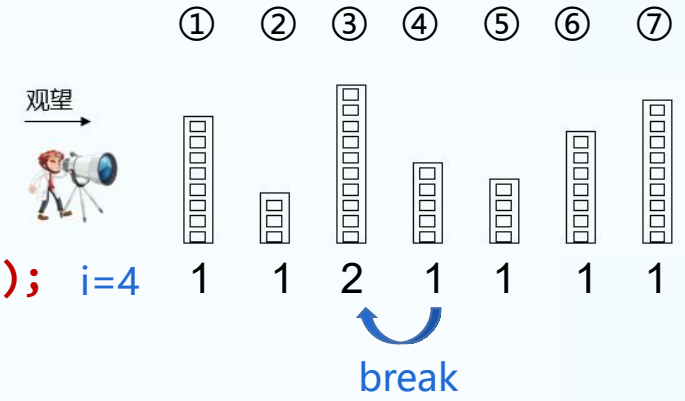
问题分析

调整“制作信箱”的状态转移代码

dp数组:存储以第i栋楼为终点的观望数量
a数组:存储楼高

```
//遍历楼高
for(int i=1; i<=n; i++){
    //遍历第i栋楼左侧所有楼高
    for(int j=i-1; j>=1; j--){
        //状态转移
        if(a[j]<a[i]) dp[i]=max(dp[i],dp[j]+1);
        else break;
    }
}
```

执行流程演示



内层循环**从右至左遍历**，当大于等于第 **i** 栋高度，结束当前循环(内层循环)

解码时刻

问题分析

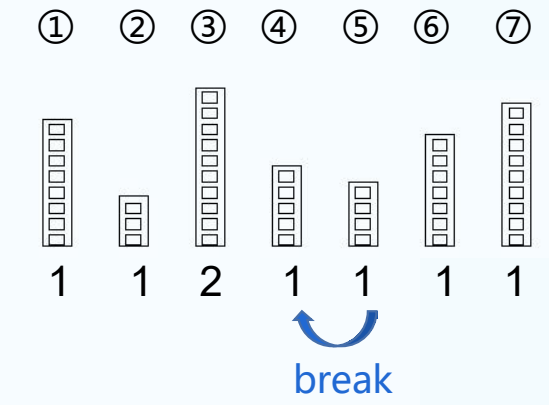
调整“制作信箱”的状态转移代码

dp数组:存储以第i栋楼为终点的观望数量
a数组:存储楼高

```
//遍历楼高
for(int i=1; i<=n; i++){
    //遍历第i栋楼左侧所有楼高
    for(int j=i-1; j>=1; j--){
        //状态转移
        if(a[j]<a[i]) dp[i]=max(dp[i],dp[j]+1);
        else break;
    }
}
```



执行流程演示



内层循环**从右至左遍历**，当大于等于第 **i** 栋高度，结束当前循环(内层循环)

解码时刻

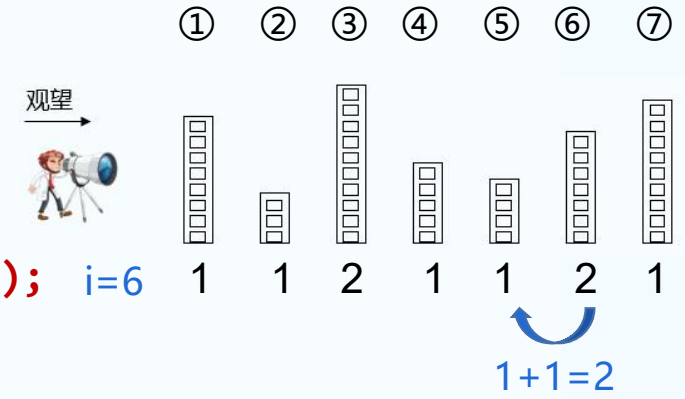
问题分析

调整“制作信箱”的状态转移代码

dp数组:存储以第i栋楼为终点的观望数量
a数组:存储楼高

```
//遍历楼高
for(int i=1; i<=n; i++){
    //遍历第i栋楼左侧所有楼高
    for(int j=i-1; j>=1; j--){
        //状态转移
        if(a[j]<a[i]) dp[i]=max(dp[i],dp[j]+1);
        else break;
    }
}
```

执行流程演示



内层循环**从右至左遍历**，当大于等于第 **i** 栋高度，结束当前循环(内层循环)

解码时刻

问题分析

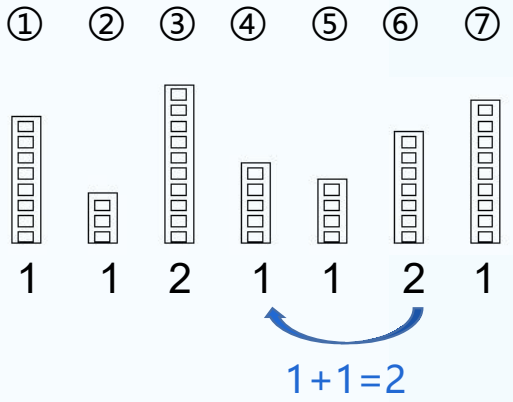
调整“制作信箱”的状态转移代码

dp数组:存储以第i栋楼为终点的观望数量
a数组:存储楼高

```
//遍历楼高
for(int i=1; i<=n; i++){
    //遍历第i栋楼左侧所有楼高
    for(int j=i-1; j>=1; j--){
        //状态转移
        if(a[j]<a[i]) dp[i]=max(dp[i],dp[j]+1);
        else break;
    }
}
```



执行流程演示



内层循环**从右至左遍历**，当大于等于第 **i** 栋高度，结束当前循环(内层循环)

解码时刻

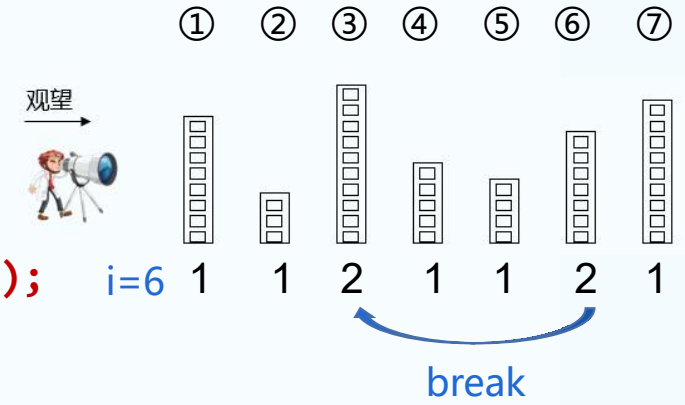
问题分析

调整“制作信箱”的状态转移代码

dp数组:存储以第i栋楼为终点的观望数量
a数组:存储楼高

```
//遍历楼高
for(int i=1; i<=n; i++){
    //遍历第i栋楼左侧所有楼高
    for(int j=i-1; j>=1; j--){
        //状态转移
        if(a[j]<a[i]) dp[i]=max(dp[i],dp[j]+1);
        else break;
    }
}
```

执行流程演示



内层循环**从右至左遍历**，当大于等于第 **i** 栋高度，结束当前循环(内层循环)

解码时刻

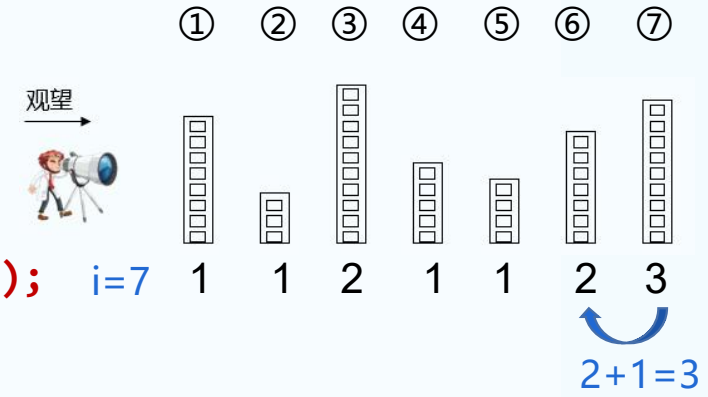
问题分析

调整“制作信箱”的状态转移代码

dp数组:存储以第i栋楼为终点的观望数量
a数组:存储楼高

```
//遍历楼高
for(int i=1; i<=n; i++){
    //遍历第i栋楼左侧所有楼高
    for(int j=i-1; j>=1; j--){
        //状态转移
        if(a[j]<a[i]) dp[i]=max(dp[i],dp[j]+1);
        else break;
    }
}
```

执行流程演示



内层循环**从右至左遍历**，当大于等于第 **i** 栋高度，结束当前循环(内层循环)

解码时刻

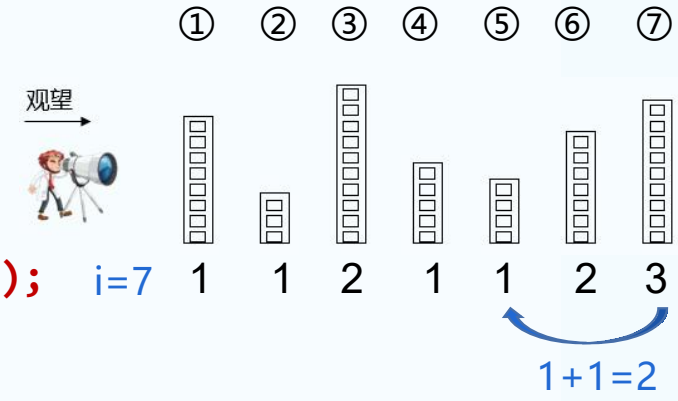
问题分析

调整“制作信箱”的状态转移代码

dp数组:存储以第i栋楼为终点的观望数量
a数组:存储楼高

```
//遍历楼高
for(int i=1; i<=n; i++){
    //遍历第i栋楼左侧所有楼高
    for(int j=i-1; j>=1; j--){
        //状态转移
        if(a[j]<a[i]) dp[i]=max(dp[i],dp[j]+1);
        else break;
    }
}
```

执行流程演示



内层循环**从右至左遍历**，当大于等于第 **i** 栋高度，结束当前循环(内层循环)

解码时刻

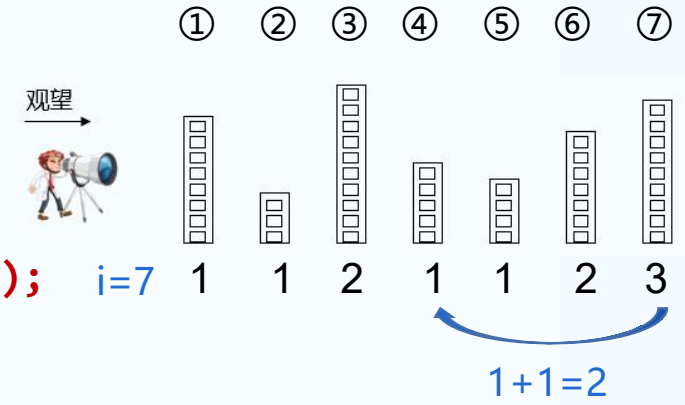
问题分析

调整“制作信箱”的状态转移代码

dp数组:存储以第i栋楼为终点的观望数量
a数组:存储楼高

```
//遍历楼高
for(int i=1; i<=n; i++){
    //遍历第i栋楼左侧所有楼高
    for(int j=i-1; j>=1; j--){
        //状态转移
        if(a[j]<a[i]) dp[i]=max(dp[i],dp[j]+1);
        else break;
    }
}
```

执行流程演示



内层循环**从右至左遍历**，当大于等于第 **i** 栋高度，结束当前循环(内层循环)

解码时刻

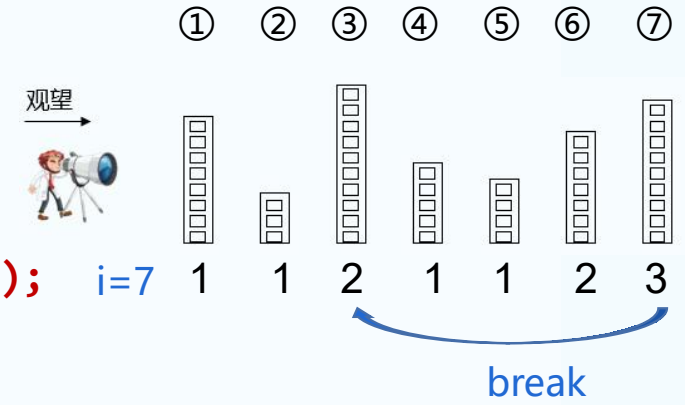
问题分析

调整“制作信箱”的状态转移代码

dp数组:存储以第i栋楼为终点的观望数量
a数组:存储楼高

```
//遍历楼高
for(int i=1; i<=n; i++){
    //遍历第i栋楼左侧所有楼高
    for(int j=i-1; j>=1; j--){
        //状态转移
        if(a[j]<a[i]) dp[i]=max(dp[i],dp[j]+1);
        else break;
    }
}
```

执行流程演示



内层循环**从右至左遍历**，当大于等于第 **i** 栋高度，结束当前循环(内层循环)



解题步骤

1.完成数据输入

```
int a[1001], n; //定义在全局,a存储楼高,n存储楼数量
//输入数量
cin>>n;
for (int i=1; i<=n; i++) {
    cin>>a[i]; //输入楼高
}
```

2.初始化状态

```
int dp[1001]; //定义在全局
for (int i=1; i<=n; i++) {
    dp[i]=1;
}
```

解题步骤

3.状态转移方程

```
for(int i=1; i<=n; i++){
    //遍历第i栋楼左侧所有楼高
    for(int j=i-1; j>=1; j--){
        //状态转移
        if(a[j]<a[i]) dp[i]=max(dp[i],dp[j]+1);
        else break;  //出现高度大于等于第i栋楼，跳出内层循环
    }
}
```

4.输出结果

```
int ans=0;  //存储结果
for(int i=1;i<=n;i++){
    ans=max(ans,dp[i]); //保存最大值
}
cout<<ans;
```

完整代码

```cpp
#include<bits/stdc++.h>
using namespace std;
int a[1001],dp[1001], n;
int main() {
    //输入
    cin>>n;
    for (int i=1; i<=n; i++) {
        cin>>a[i];
    }
    //初始化
    for (int i=1; i<=n; i++) {
        dp[i]=1;
    }
    for(int i=1; i<=n; i++){
        for(int j=i-1; j>=1; j--){
            //状态转移
            if(a[j]<a[i]) {
                dp[i]=max(dp[i],dp[j]+1);
            }
            else break;
        }
    }
    int ans=0;
    for(int i=1;i<=n;i++) {
        ans=max(ans,dp[i]);
    }
    //输出结果
    cout<<ans;
    return 0;
}
```